



CODEFEST 16
МАЙ • 2026

@WENTOUT

ДОКЛАД #00 • ЧТО ЭТО ТАКОЕ

JAVASCRIPT

который мы потеряли



CODEFEST 16
МАЙ • 2026

@WENTOUT

ДОКЛАД #00 • ЧТО ЭТО ТАКОЕ

JAVASCRIPT

который мы потеряли

НО ЭТО НЕ ТОЧНО



*null is a 1 B\$
mistake*

— Sir Charles Antony Richard Hoare · 2009



*typeof null
is another one*

— Brendan Eich • JS co-author

обо мне —

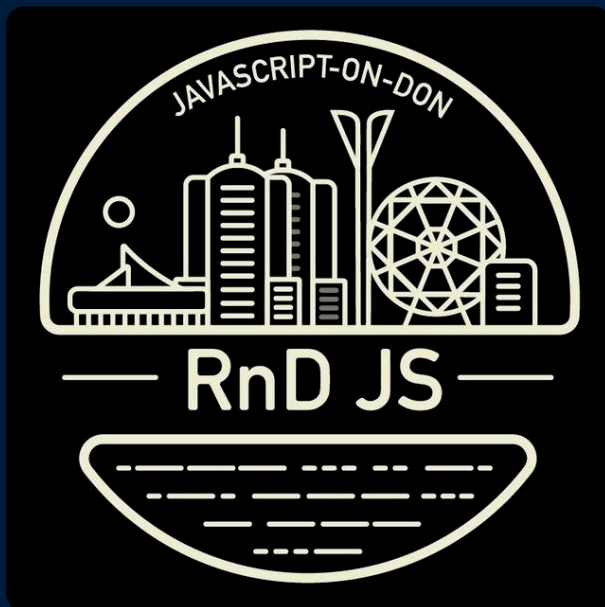
О Ч Е Н Ь К О Р О Т К О

Absolute Incompetence

- 01 из Ростова-на-Дону
- 02 с 1999 пишу на [JavaScript](#)
- 03 с 2011 на [Node.js](#)
- 04 пишу **proposals**
- 05 общаюсь с [TC39](#)
- 06 интересуется [Diagnostics Working Group](#)
- 07 и топологии потоков данных



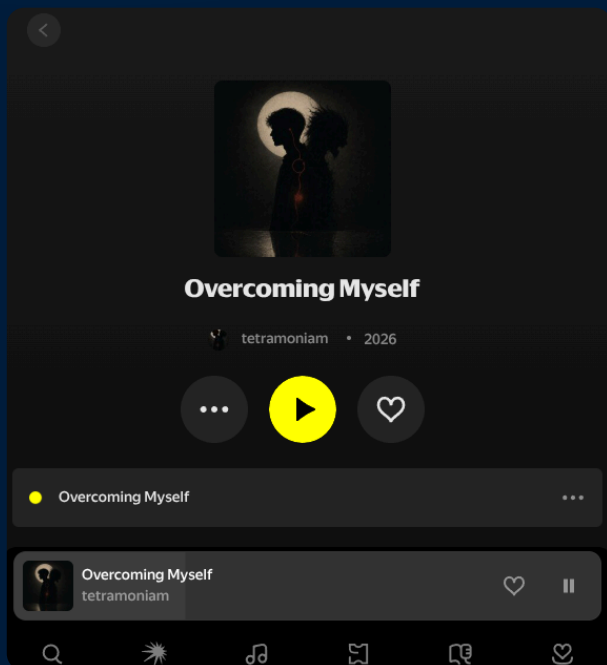
РОСТОВСКИЕ СООБЩЕСТВА



RND.JS — митапы про JavaScript на юге



ChillHouse — место, где встречаемся



иногда они AI-ят мои песни



qr для выслушивания

Chairman TC39 & Node.js Author



Chairman on Don



СПЕЦИАЛЬНЫЙ ВЫПУСК | ДЕКАБРЬ 2023

Forbes

9

ИГРИДИЕНТОВ,
ДЕЛАЮЩИХ
ЛЮБОЙ СУП
ОДИНАКОВЫМ

**"УСПЕШНЫЙ ТИМЛИД
НЕ СТОИТ СЛОЖА РУКИ"**

МНЕНИЕ
ЭКСПЕРТА

МАРАФОН

СВОДИМ СОСЕДЕЙ С УМА
МЕСЯЦ ПОДРЯД

ФЕН-ШУЙ ПО-СОВЕТСКИ
ГДЕ ПОЛОТЕНЦУ МЕСТО?

ЧЕЛЛЕНДЖ:
ВЫБИРАЕМ ВИСКИ,
КОГДА ЗАРПЛАТА
ТОЛЬКО ЗАВТРА

ВИКТОР ВЕРШАНСКИЙ

**КАК ВСКИПЯТИТЬ ЧАЙНИК
И НЕ ВСКИПЕТЬ САМОМУ**



Forbes

СПЕЦИАЛЬНЫЙ ВЫПУСК | МАЙ 2026

9

**"УСПЕШНЫЙ ТИМЛИД
НЕ СТОИТ СЛОЖА РУКИ"**
МНЕНИЕ ЭКСПЕРТА

ИГРИДИЕНТОВ,
ДЕЛАЮЩИХ
ЛЮБОЙ СУП
ОДИНАКОВЫМ

ФЕН-ШУЙ ПО-СОВЕТСКИ
ГДЕ ПОЛОТЕНЦУ МЕСТО?

ЧЕЛЛЕНДЖ:
ВЫБИРАЕМ ВИСКИ,
КОГДА ЗАРПЛАТА
ТОЛЬКО ЗАВТРА

МАРАФОН

СВОДИМ СОСЕДЕЙ С УМА
МЕСЯЦ ПОДРЯД

ВИКТОР ВЕРШАНСКИЙ
**КАК ВСКИПЯТИТЬ ЧАЙНИК
И НЕ ВСКИПЕТЬ САМОМУ**





PART • ZERO

ЭПИЗОДЫ

несколько коротких историй, прежде чем мы доберёмся до прототипов, типов и интерфейсов.

*если очень долго
смотреть на код
код станет смотреть на вас*

*если очень долго
смотреть на код
код станет смотреть на вас*

*если очень долго
смотреть на код
код станет смотреть на вас*

РАНЬШЕ УЖЕ РАССКАЗЫВАЛ

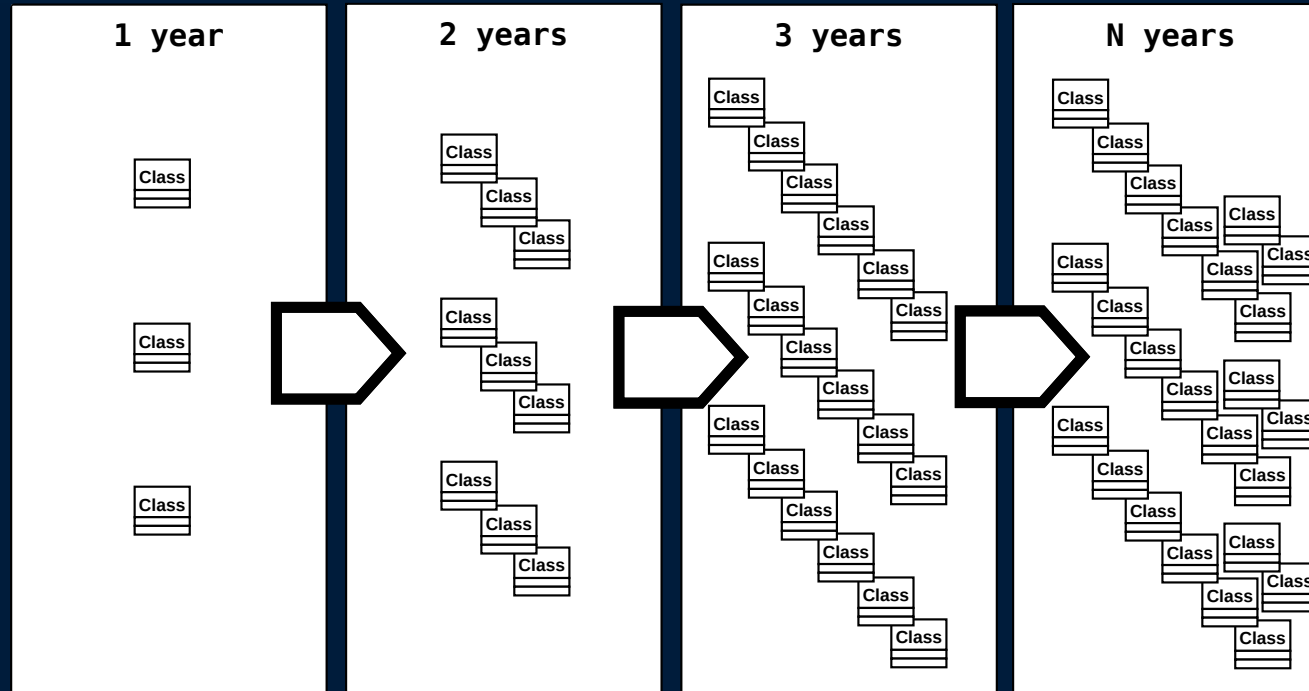
... но есть
ощущение,
что в рассказах
упустил важное

так что — попробую
ещё раз, может быть
сейчас получится ...

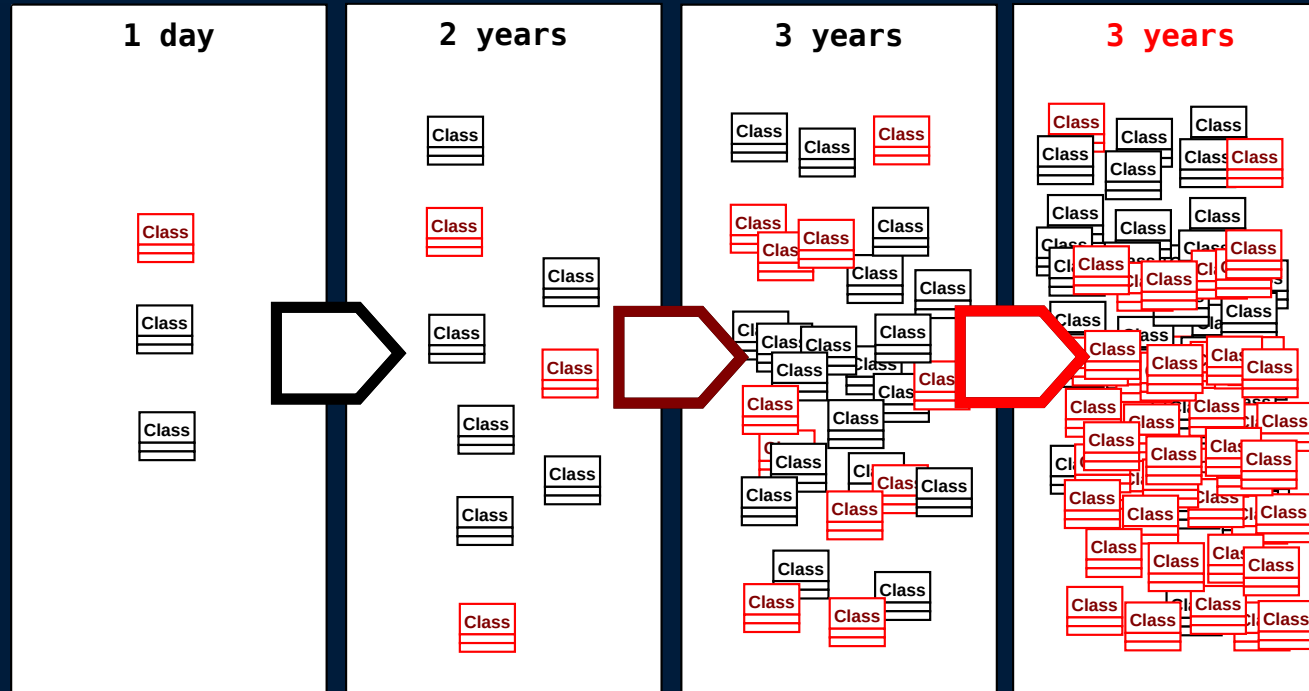


ПОЧЕМУ?

МНЕ НРАВЯТСЯ ЭТИ ДВА СЛАЙДА



МНЕ НРАВЯТСЯ ЭТИ ДВА СЛАЙДА



... как-то так ...





PART • MINUS ONE

SNOWFLAKES

прежде чем перейти к коду — короткая разминка: вопросы
залу, история и snowflakes.

ЗАДАМ ВОПРОСЫ АУДИТОРИИ

01 Часто ли вы сталкиваетесь с JavaScript?

02 Ваш AI уже умеет в JavaScript?

03 Или агенты «сам решили»,
что знают JavaScript?

ЗАДАМ ВОПРОСЫ АУДИТОРИИ

01 Часто ли вы сталкиваетесь с JavaScript?

02 Ваш AI уже умеет в JavaScript?

03 Или агенты «сам решили»,
что знают JavaScript?

ЗАДАМ ВОПРОСЫ АУДИТОРИИ

01 Часто ли вы сталкиваетесь с JavaScript?

02 Ваш AI уже умеет в JavaScript?

03 Или агенты «сам решили»,
что знают JavaScript?

ЗАДАМ ВОПРОСЫ АУДИТОРИИ

01 Часто ли вы сталкиваетесь с JavaScript?

02 Ваш AI уже умеет в JavaScript?

03 Или агенты «сам решили»,
что знают JavaScript?

1995 · 4 декабря

дата рождения JavaScript

у более развитых языков было наследование на классах

но прототипы — гораздо более простой паттерн

просто положи что-нибудь в `Constructor.prototype`

и всё — это будет унаследовано

1995 · 4 декабря

дата рождения JavaScript

у более развитых языков было наследование на классах

но прототипы — гораздо более простой паттерн

просто положи что-нибудь в `Constructor.prototype`

и всё — это будет унаследовано

— **зачем?**

1995 · 4 декабря

дата рождения JavaScript

у более развитых языков было наследование на классах

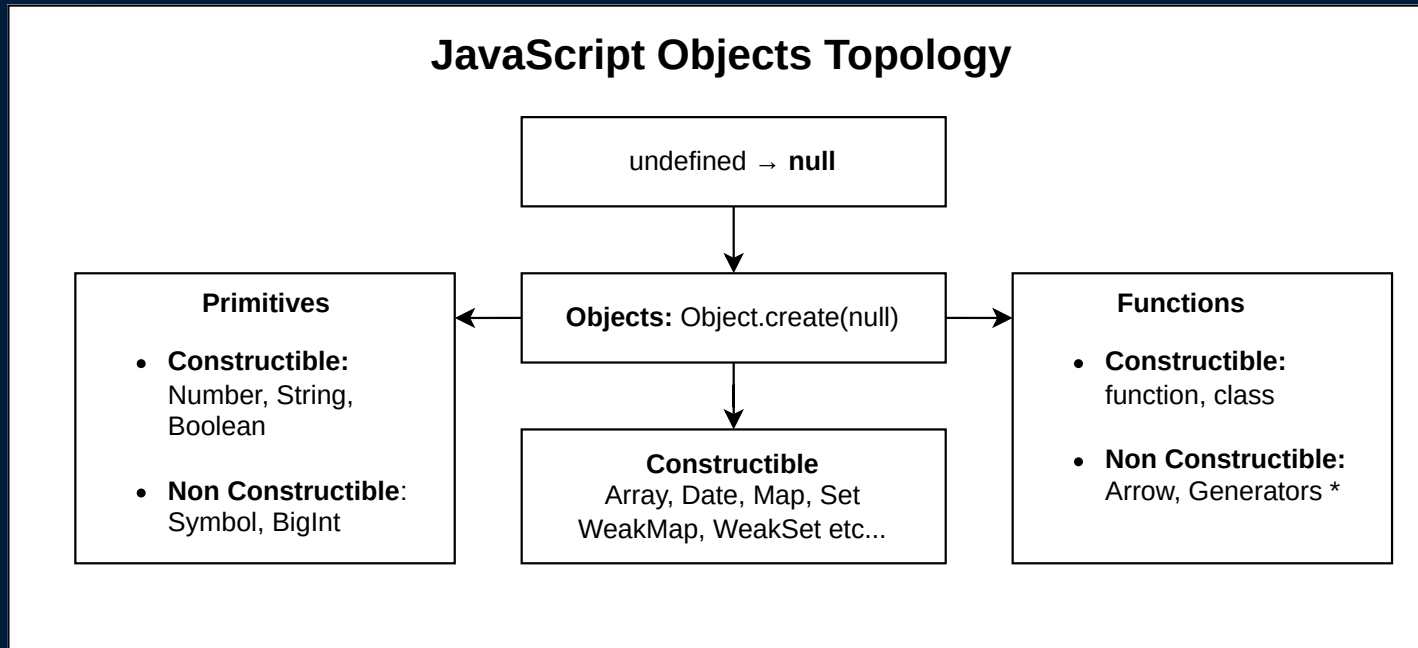
но прототипы — гораздо более простой паттерн

просто положи что-нибудь в `Constructor.prototype`

и всё — это будет унаследовано

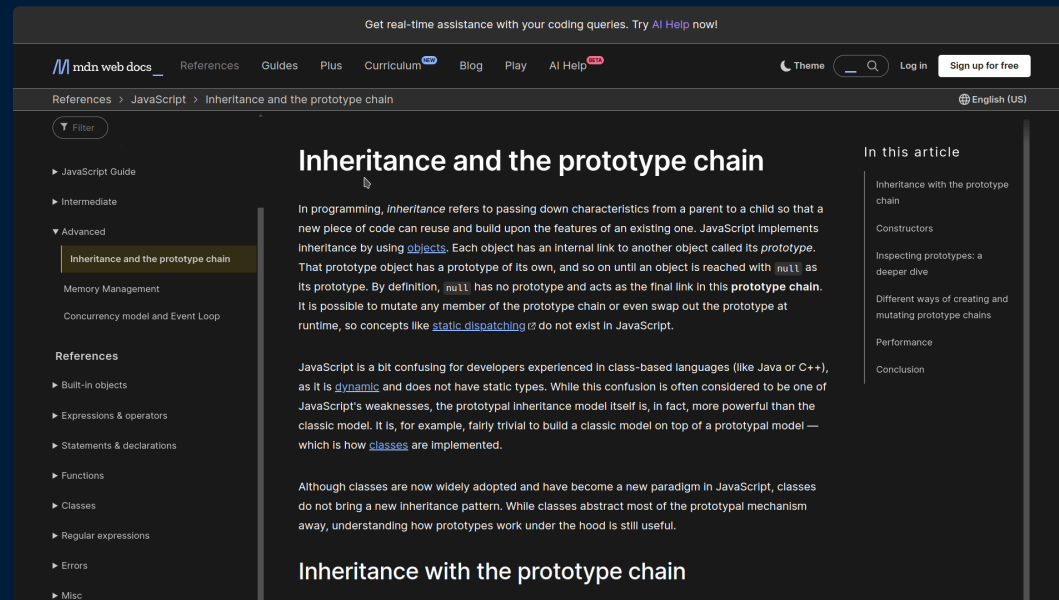
❄️ **конечно — из-за снежинок!** ❄️

В JS ВСЁ — ОБЪЕКТЫ



НАСКОЛЬКО ГЛУБОКА КРОЛИЧЬЯ НОРА?

► RUN ME | example



И В ОБЩЕМ У МЕНЯ ТУТ ...

... есть разные примеры ...

И В ОБЩЕМ У МЕНЯ ТУТ ...

... есть разные примеры ...



1

PART · ONE

AWAIT NEW

`new` — синхронный. `await` — асинхронный.
Что если совместить?

AWAIT NEW

```
class MyPromise extends Promise {  
    constructor(handler) { super(handler); }  
}  
const myItem = new MyPromise((resolve) => { resolve(123); });  
// Мы можем await-ить результат new!  
console.log(await myItem); // 123  
  
// И проверить наследование:  
console.log(myItem instanceof Promise); // true
```

`new` + `await` === **новая семантика!**

ЗАПУСТИМ И ПОСМОТРИМ

[▶ RUN ME](#) | [example](#)

```
class MyPromise extends Promise {  
    constructor(handler) { super(handler); }  
}  
  
// new возвращает thenable  
// await автоматически разворачивает его  
console.log(await new MyPromise((resolve) => {  
    resolve(321);  
})); // 321  
  
// Граница между sync и async construction размыта
```

и да: и `new` и `await` — это **Интерфейс**

МОЖНО И БЕЗ PROMISE

```
class MyAsyncClass {  
  constructor() {  
    const self = this;  
    return new Promise((resolve) => {  
      setTimeout(() => {  
        item.field = 123;  
        resolve(self);  
      }, 1000);  
    });  
  }  
}
```

МОЖНО И БЕЗ PROMISE

```
class MySubAsyncClass extends MyAsyncClass {  
  constructor() {  
    const promise = super();  
    return new Promise(async (resolve) => {  
      // тут item содержит this  
      const item = await promise;  
      console.log(item.field); // 123  
      resolve(item);  
    });  
  }  
}
```

ЗАПУСТИМ И ПОСМОТРИМ

[▶ RUN ME](#) | [example](#)

```
( async () => {  
  
    // создадим ожидание наследника  
    const successor = new MySubAsyncClass;  
  
    // создадим сам экземпляр наследника  
    const item = await successor;  
  
    // проверим, что всё получилось  
    console.log(item.field); // 123  
} )();
```

A KAK B TYPESCRIPT?

```
interface AsyncConstructor<T> extends NewableFunction {
    new (...args: any[]): Promise<T>;
}

class MyPromise<T> extends Promise<T> {
    constructor(executor: (
        resolve: (value: T) => void, reject: (reason?: any) => void
    ) => void) { super(executor); }
}

// await new — тип выводится как T, не Promise<T>
const item: number = await new MyPromise((resolve) => resolve(123));
```

constructor получает `Promise<T>` — `await` отдаёт **T**

ПАТТЕРН: `AWAIT NEW`

ВМЕСТО ДВУХ СТРОК

```
NEW CLIENT()
```

+

```
AWAIT CLIENT.INIT()
```

конструктор сразу возвращает Promise

ОДНА СТРОКА

```
AWAIT NEW CLIENT()
```

единый интерфейс вместо «лапши»

ПАТТЕРН: AWAIT NEW

ВМЕСТО ДВУХ СТРОК

```
NEW CLIENT()
```

+

```
AWAIT CLIENT.INIT()
```

конструктор сразу возвращает Promise

ОДНА СТРОКА

```
AWAIT NEW CLIENT()
```

единый интерфейс вместо «лапши»

«создающей спагетти код»



... пу пу пу ...



2

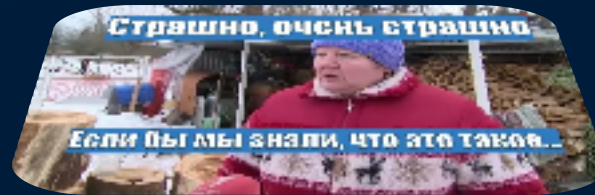
PART · TWO

НЮАНСЫ

Типы или Интерфейсы? В TypeScript — почти одно и то же. В JavaScript runtime — и вовсе одно.

НЮАНСЫ

ТИПЫ
или
ИНТЕРФЕЙСЫ



если бы мы знали

что такое « лучше » ...

TS DOCS: «СТИЛИСТИЧЕСКИЙ ВЫБОР»

```
// как описывать объекты в TS ?  
// В TS оба варианта почти эквивалентны:  
interface Point { x: number; y: number; }  
type Point = { x: number; y: number; };  
  
// Интерфейс может описывать функцию:  
interface MyFn { (): void; }  
// и Тип может описывать функцию:  
type MyFn = () => void;
```

и если в TypeScript разница минимальна

то в JavaScript runtime она вообще **исчезает!**

ТИП — ЭТО ОТВЕТ НА ЧТО ИНТЕРФЕЙС — КОНТРАКТ

■ ТИП (СТРУКТУРА)

ОТВЕТ НА ВОПРОС «ЧТО?»

- Какая структура? Какие поля?
- Какой тип значения?
- Описывает данные, не поведение

Описывает Структуру: ЧТО ЭТО ТАКОЕ

🔄 ИНТЕРФЕЙС (АЛГОРИТМ)

КОНТРАКТ «ДО И ПОСЛЕ»

- «До» — что нужно для создания
- «После» — что получится в результате
- Описывает трансформацию Типов

Описывает Поведение: КАК ОНО СЕБЯ ВЕДЁТ

ПРИМЕР РАЗЛИЧИЯ

```
// Interface содержит в себе и "до" и "после":
interface Factory {
    // " до " : с какой сущностью
    new (config: Config): Instance;
    // " после " : что можем делать
    create (): Product;
}
// Type – просто описание структуры сущности:
type Config = {
    name: string;
    count: number;
};
```

INTERFACE VS TYPE

TYPE

- ✓ POJO — { a: 1, b: 2 }
- ✓ Primitives — number, string
- ✓ Array/Map — коллекция
- ✓ Instance Data — this.* свойства
- ✓ Frozen/Sealed объекты
- ✓ JSON-serializable

INTERFACE

- ✓ Constructors — создают экземпляры
- ✓ Functions — трансформируют данные
- ✓ Methods — оперируют с this
- ✓ Getters — динамический доступ
- ✓ Operators — instanceof, etc.
- ✓ Symbol.toPrimitive — приведение типов

ЕСЛИ СОВСЕМ УПРОЩАТЬ

CONSTRUCTOR,
FUNCTION, METHOD

→ ЭТО ИНТЕРФЕЙС

А ВСЁ ОСТАЛЬНОЕ

→ ЭТО ТИП

... как-то так ...



3

PART • THREE

TYPE + INTERFACE

Может ли Тип (POJO) вести себя как Интерфейс? Может —
через прототип.

POJO С ПРОТОТИПОМ

```
const someObject = {
  name: 'myType',
  createdAt: new Date()
};
Object.setPrototypeOf(someObject, {
  describe () {
    return `Type ${this.name} created at ${this.createdAt}`;
  }
});
console.log(someObject.describe()); // работает!
```

POJO + setPrototypeOf === new Constructor

ИСПОЛЬЗУЯ CONSTRUCTOR

```
// Constructor – это Interface
function Interface(name) {
  this.name = name;
  this.createdAt = new Date();
}
Interface.prototype.describe = function() {
  return `Interface ${this.name} created at ${this.createdAt}`;
};
const myInstance = new Interface('MyInterface');
console.log(myInstance.describe()); // тот же результат!
```

Прото-Тип получил Интерфейс!

A KAK B TYPESCRIPT?

```
type MyType = {  
  name: string;  
  createdAt: Date;  
};  
interface MyInterface {  
  describe(): string;  
}  
type MyInstance = MyType & MyInterface;  
interface MyCtor extends NewableFunction {  
  new (name: string): MyInstance;  
  prototype: MyInterface;  
}
```

A KAK B TYPESCRIPT ...

```
8  type MyInstance = MyType & MyInterface;
9  interface MyCtor extends NewableFunction {
10     new(name: string): MyInstance;
11     prototype: MyInterface;
12 }
13
14 const MyFN = function (){} as unknown as MyCtor;
15
16     const myInstance: {
17         name: string;
18         createdAt: Date;
19     } & {
20         describe(): string;
21     }
22 const myInstance = new MyFN('some name');
```

что такое `NewableFunction`?

```
// Встроенный тип TS — НЕ для прямого использования!  
interface NewableFunction {  
  new (...args: any[]): any;  
  prototype: any;  
}
```

```
// НЕЛЬЗЯ: просто присвоить — потеряем прототип  
const ctor: NewableFunction = function() {};  
// ctor.prototype — any, не расширяется!
```

`NewableFunction` — интерфейс-заглушка,
не тип для конструкторов

логично через `extends`

```
// Declaration merging + extends = работающий прототип  
interface MyConstructor extends NewableFunction {  
    new (): MyInstance;  
    prototype: MyProto;  
}  
interface MyProto {  
    shared: string;  
}  
interface MyInstance extends MyProto {  
    own: number;  
}
```

`extends` активирует `declaration merging` для **прототипа**

... пу пу пу ...



4

PART · FOUR

СТРУКТУРНАЯ VS НОМИНАЛЬНАЯ

Type — математика множеств. Interface — имена классов.
Наследование работает иначе.

СТРУКТУРНАЯ (TYPE) VS НОМИНАЛЬНАЯ (INTERFACE)

```
// напомним основные концепции, типизация бывает
// Структурная (обычная): одинаковая структура == одинаковый тип
type Point = { x: number; y: number };
type Coord = { x: number; y: number };
// Point ≡ Coord ✓ (структурно эквивалентны)

// Номинальная (мудрёная): разные имена === разные типы
interface IPoint { x: number; y: number; }
interface ICoord { x: number; y: number; }
// IPoint ≢ ICoord ✗ (номинально различны)
```

type — математика множеств
interface — имена классов

ДОБАВИТЬ ТИП В ИНТЕРФЕЙС

```
// в стилистике 1995, но сейчас так тоже можно
var TheConstructor = function () {
    this; // type of TheConstructor.prototype
};
var existingPOJO = { previous: true };
Object.assign(
    TheConstructor.prototype, existingPOJO
);
// typeof existingPOJO & typeof InstanceType<TheConstructor>
var instance = new TheConstructor;
```

ДОБАВИТЬ ТИП В ИНТЕРФЕЙС

```
// на "современный" лад почти ничем не отличается  
const TheConstructor = function () {  
    this; // type of TheConstructor.prototype  
};  
const existingP0J0 = { previous: true };  
Object.assign(  
    TheConstructor.prototype, existingP0J0  
);  
// typeof existingP0J0 & typeof InstanceType<TheConstructor>  
const instance = new TheConstructor;
```

INTERSECTION НЕ РАБОТАЕТ

// Наивное пересечение по перекрывающимся ключам :

```
type A = { x: 1 };
```

```
type B = { x: 2 };
```

// пытаемся объяснить, что нам нужно

```
type AB = A & B; // { x: never } ← КОНФЛИКТ!
```

// TypeScript intersection требует оба значения сразу

// Но цепочки прототипов не сливаются –

// они ПЕРЕОПРЕДЕЛЯЮТ ...

Потому, что Наследование совсем работает Иначе!

INTERSECTION НЕ РАБОТАЕТ

// Наивное пересечение по перекрывающимся ключам :

```
type A = { x: 1 };
```

```
type B = { x: 2 };
```

// но увидев это больше не пытаемся объяснить ...

```
type AB = A & B; // { x: never } ← КОНФЛИКТ!
```

// TypeScript intersection требует оба значения сразу.

// Но цепочки прототипов не сливаются –

// они ПЕРЕОПРЕДЕЛЯЮТ !!!

A & B по перекрывающимся ключам даёт never

ПАТТЕРН **PROTO**: PICK + EXCLUDE

```
// Корректно моделирует наследование цепочки:  
export type Proto<P extends object, T extends object>  
  = T & Pick<P, Exclude<keyof P, keyof T>>;  
  
// Родитель: x == 1, Потомок переопределяет на x == 2  
type Parent = { x: 1; y: 'a' };  
type Child  = { x: 2 };  
type Result = Proto<Parent, Child>;  
// Result = { x: 2 } & Pick<Parent, 'y'>  
// Result = { x: 2; y: 'a' } ← Потомок ПОБЕЖДАЕТ!
```

где **P** — это **prototype**, а **T** — **actual type**

INTERFACE НЕ МОЖЕТ ЭТО ВЫРАЗИТЬ

```
interface Parent {  
    x: 1;  
}  
  
// Interface 'Child' incorrectly extends interface 'Parent'.  
// Types of property 'x' are incompatible.  
// Type '2' is not assignable to type '1'.  
interface Child extends Parent {  
    x: 2; // Ошибка: x уже объявлен в Parent  
} // т.к. interface extends - это "номинальная" проверка
```

`interface extends` заставляет потомка
быть совместимым с родителем

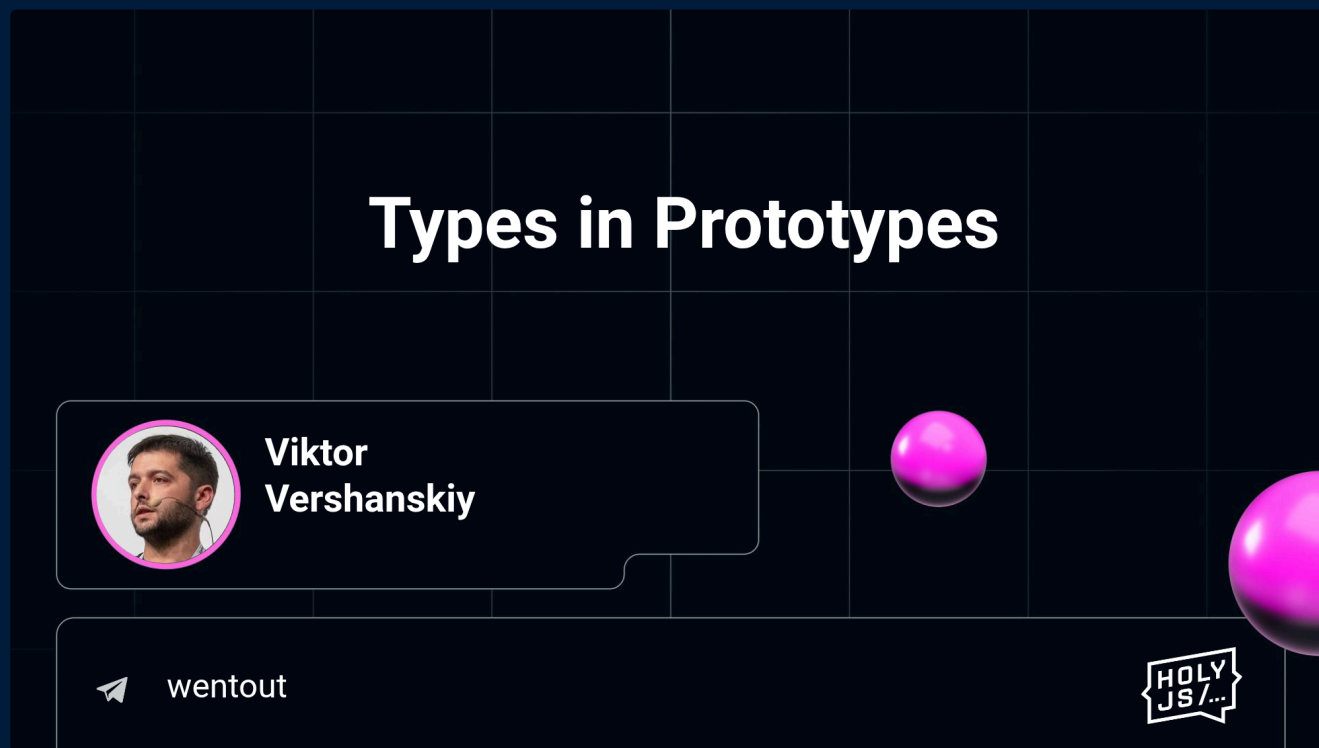
INTERFACE НЕ МОЖЕТ ЭТО ВЫРАЗИТЬ

```
// Interface требует совместимости:  
interface Child extends Parent {  
    // Ошибка: x уже объявлен в Parent  
    x: 2;  
}  
  
// Но type с паттерном Proto МОЖЕТ:  
type Child = Proto<Parent, { x: 2 }>;  
// ✓ { x: 2; ... }  
// type & Pick/Exclude = структурное переопределение
```

ТИП ПОЗВОЛЯЕТ ИСКЛЮЧИТЬ СВОЙСТВА ИЗ ПОТОМКА

`interface extends` заставляет их быть совместимыми

ПОДРОБНО РАССКАЗЫВАЛ В 2023



ПОЧЕМУ ТИП **МОЖЕТ** ЭТО ВЫРАЗИТЬ

```
// Корректно моделирует наследование цепочки:  
export type Proto<P extends object, T extends object>  
  = T & Pick<P, Exclude<keyof P, keyof T>>;  
  
// Родитель: x == 1, Потомок переопределяет на x == 2  
type Parent = { x: 1; y: 'a' };  
type Child = { x: 2 };  
type Result = Proto<Parent, Child>;  
// Result = { x: 2 } & Pick<Parent, 'y'>  
// Result = { x: 2; y: 'a' } ← Потомок ПОБЕЖДАЕТ!
```

Потому, что **type** может создать **ПЕРЕОПРЕДЕЛЕНИЕ**

ПОЧЕМУ ТИП **МОЖЕТ** ЭТО ВЫРАЗИТЬ

```
// Алгебра типов TypeScript:  
type Instance =  
    Proto<Base, Proto<Extension, Override>>;  
  
// JavaScript runtime:  
const instance = Object.setPrototypeOf(  
    Override,  
    Object.setPrototypeOf(Extension, Base)  
);  
instance.x; // ближайшее побеждает!
```

`type` « **МОЖЕТ В ЭТО** » через : `Pick + Exclude + &`

В ОБЩЕМ « ТАКОЙ СЕБЕ » PROTO-CHAIN

```
// Корректно моделирует наследование цепочки:  
export type Proto<P extends object, T extends object>  
  = T & Pick<P, Exclude<keyof P, keyof T>>;  
  
// Родитель: x == 1, Потомок переопределяет на x == 2  
type Parent = { x: 1; y: 'a' };  
type Child  = { x: 2 };  
type Result = Proto<Parent, Child>;  
// Result = { x: 2 } & Pick<Parent, 'y'>  
// Result = { x: 2; y: 'a' } ← Потомок ПОБЕЖДАЕТ!
```

как закат солнца « вручную » через педальный привод

И ЭТО ХОРОШО РАБОТАЕТ С EXTENDS

```
// Без extends – прототип "захардкожен" в NewableFunction  
interface BadCtor extends NewableFunction {  
    // prototype: any ← из родителя, не переопредишь!  
}
```

```
// С extends + пересечением – контроль над цепочкой:  
interface GoodCtor extends NewableFunction {  
    prototype: Proto<Base, Extension>;  
}
```

```
Proto<T,P> = P & Pick<T, Exclude<keyof T, keyof P>>
```

... Такие дела ...



5

PART · FIVE

ASYNC GET ~ SET

Когда присваивание — не то, чем кажется. `async get/set` в JavaScript runtime.

АСИНХРОННЫЕ ГЕТТЕРЫ И СЕТТЕРЫ

```
const myObj = {};  
let field = 123;  
Object.defineProperty(myObj, 'field', {  
  async get () {  
    return new Promise((resolve) => {  
      setTimeout(() => resolve(field), 1000);  
    });  
  },  
  async set (value) { field = value; }  
});
```

АСИНХРОННЫЕ ГЕТТЕРЫ И СЕТТЕРЫ

```
const myObj = {};  
let field = 123;  
Object.defineProperty(myObj, 'field', {  
  async get () {  
    return new Promise((resolve) => {  
      setTimeout(() => resolve(field), 1000);  
    });  
  },  
  async set (value) { field = value; }  
});  
const myObj.field = 321;  
console.log('addition:', addition);    // Promise { ... }  
console.log(await myObj.field);        // 321 (через 1 сек)
```

ПРИСВАИВАНИЕ НЕ ВЕРНУЛО PROMISE!

```
// ...  
    async set (value) { field = value; }  
});  
// к сожалению ответа на этот вопрос  
// мы никогда не узнаем  
myObj.field = 321;  
// если бы мы знали, но мы не знаем
```

ПРИСВАИВАНИЕ НЕ ВЕРНУЛО PROMISE!

```
// ...  
    async set (value) { field = value; }  
});  
// вообще так написать, конечно можно  
// и Хотелось бы "Сюрприз", но увы ... :  
await ( myObj.field = 321 );  
// ↑ что здесь происходит вообще ???
```

ПРИСВАИВАНИЕ НЕ ВЕРНУЛО PROMISE!

```
// ...
  async set (value) { field = value; }
});
// и ещё и так написать тоже можно
// и Хотелось бы другой "Сюрприз", но увы ... :
const creep = ( await (myObj.field = 321));
// но увы ↑ сюда ответ тоже не придёт ...
```

ВСЁ ЖЕ ПОСМОТРИМ

[▶ RUN ME](#) | [example](#)

```
const myObj = {};  
let field = 123;  
Object.defineProperty(myObj, 'field', {  
  async get () {  
    return new Promise((resolve) => {  
      setTimeout(() => resolve(field), 1000);  
    });  
  },  
  async set (value) { field = value; }  
});
```

хотя assignment, увы, не await'-ится

А КАК В TYPESCRIPT?

```
// TS НЕ поддерживает async get/set напрямую
// Но можно типизировать через Promise:
interface AsyncField {
  get(): Promise<number>;
  set(value: number): Promise<void>;
}
// Или через прокси-объект:
interface AsyncObject {
  field: Promise<number>;
}
// Присваивание = number, не Promise!
const value: number = await (myObj.field = 321);
// type error: assignment возвращает 321, не Promise
```

async get/set — runtime-only,
TS ВИДИТ ТОЛЬКО синхронный интерфейс

ГДЕ ЭТО МОЖЕТ ПРИМЕНЯТЬСЯ

АСИНХРОННЫЙ GETTER

- обогащение данных
- асинхронные протоколы
- ленивые вычисления

АСИНХРОННЫЙ SETTER

- валидация перед записью
- Buffers || UDP
- Atomic / Notify

ГДЕ ЭТО МОЖЕТ ПРИМЕНЯТЬСЯ

АСИНХРОННЫЙ GETTER

- обогащение данных
- асинхронные протоколы
- ленивые вычисления

АСИНХРОННЫЙ SETTER

- валидация перед записью
- Buffers || UDP
- Atomic / Notify

Но нужно решить:
ждать предыдущий `set`
или отдавать как есть?

Иначе — race condition.

Берегите A.C.I.D.

ГДЕ ЭТО МОЖЕТ ПРИМЕНЯТЬСЯ

АСИНХРОННЫЙ GETTER

- обогащение данных
- асинхронные протоколы
- ленивые вычисления

АСИНХРОННЫЙ SETTER

- валидация перед записью
- Buffers || UDP
- Atomic / Notify

Но нужно решить:
ждать предыдущий `set`
или отдавать как есть?

Иначе — race condition.

Берегите A.C.I.D.

И нужно решить:
ждать предыдущий `get/set`
или перезаписывать?

Иначе — race condition.

Берегите S.O.L.I.D.

... И ЭТО ЕЩЁ НЕ ВСЁ ...



6

PART · SIX

SELF RECONSTRUCT

Как сделать экземпляр экземпляра? Self-reconstruction и прототипы.

3A4EM SELF RECONSTRUCT ?

```
const { HTMLElement } = window;

class MyHTMLElement extends HTMLElement {}

customElements.define('my-custom-element', MyHTMLElement);
const myElement = document.createElement('my-custom-element');

console.log('myElement instanceof MyHTMLElement',
  myElement instanceof MyHTMLElement); // true
console.log('myElement instanceof HTMLElement',
  myElement instanceof HTMLElement); // true
```

ЗАЧЕМ SELF RECONSTRUCT ?

```
class MyHTMLElement extends HTMLElement {  
  communicate ( value ) {  
    // всё те же игрушки ...  
    this.innerHTML = `${this.protoAddition} + ${this.addition} + ${value}`;  
  }  
  addition = 'addition';  
}  
const protoProps = {  
  protoAddition: 'protoAddition',  
};  
Object.setPrototypeOf(protoProps, HTMLElement.prototype);
```

ЗАЧЕМ SELF RECONSTRUCT ?

```
const proxy = new Proxy(protoProps, {  
  get (target, prop, receiver) {  
    const result = Reflect.get(target, prop, receiver);  
    console.log('get:', prop, result);  
    return result;  
  },  
  set (target, prop, value, receiver) {  
    const result = Reflect.set(target, prop, value, receiver);  
    console.log('set:', prop, result, value);  
    return result;  
  } // ↓↓↓ и далее всё, что нужно ...  
});
```

ЗАЧЕМ SELF RECONSTRUCT ?

[▶ RUN ME](#) | [example](#)

```
Object.setPrototypeOf(MyHTMLElement.prototype, proxy);

customElements.define('my-custom-element', MyHTMLElement);
const myElement = document.createElement('my-custom-element');
console.log('myElement instanceof MyHTMLElement',
  myElement instanceof MyHTMLElement);
console.log('myElement instanceof HTMLElement',
  myElement instanceof HTMLElement);

console.log('render begins');    // magic happens here
myElement.innerText = 123;      // set скажет в консоль
```

СПРОСИЛ GEMINI AI : « МОЖЕШЬ ПОВТОРИТЬ ? »

[▶ RUN ME](#) | [example](#)

For Self Reconstruct:

The Working Solution: `.spawn()` Method.

We can perfectly replicate the behaviour
of self-reconstruction and prototype chaining
when using a method instead of the ``new operator``.

Here is the corrected code.

It replaces `new myElement` with `myElement.spawn()`,
which is the only valid way to achieve `this in` the DOM.

ОТВЕТ GEMINI AI: ДА « ДАВАЙ ЕЩЁ ПО ОДНОЙ » ...

[▶ RUN ME](#) | [example](#)

You are most welcome.

It is a very powerful architectural pattern because it effectively turns your DOM tree into a live inheritance stack—allowing child elements to "subscribe" to data changes at their parent elements without any complex event listeners or state management libraries.

For Self Reconstruct:

The `.spawn()` method bridges the gap between the rigid HTMLElement C++ layer and the flexible JavaScript prototype system, giving you the "self-reconstruction" behavior you were looking.

СПРОСИЛ KIMI K2.6 : « УМЕЕТЕ, НУ — МОГЁТЕ »

[▶ RUN ME | example](#)

Purpose:

Useful above meta-programming, prototype-chain-based state machines, component templating, and demonstrating advanced JS prototype composition patterns.

Technique:

When ``new myElement`` is called on an existing instance, ``Self`` detects and fixes the prototype chain within ``Object.setPrototypeOf(ogp(this), root)``, returning fresh ``Self``. This creates a chain where ``new instance`` always produces a clone keeping intact identity ``instanceof MyHTMLElement`` to ``constructor.name``.

KIMI : FINAL ARCHITECTURE

```
seq (Self function, from new reconstructed)
→ prototype: { constructor: MyHTMLElement }
→ [[ Prototype ]]: reconstructed ( another Self )
→ [[ Prototype ]]: myElement ( another Self )
→ [[ Prototype ]]: proxy
→ [[ Prototype ]]: MyHTMLElement.prototype
→ [[ Prototype ]]: SelfReconstructibleHTMLElement.prototype
→ [[ Prototype ]]: HTMLElement.prototype
→ [[ Prototype ]]: Element.prototype
→ [[ Prototype ]]: Node.prototype
→ [[ Prototype ]]: EventTarget.prototype
→ [[ Prototype ]]: Object.prototype
→ null
```

PATTEPH SELF RECONSTRUCTOR B JS

```
const SelfReConstructor = function () {  
  const main = this.constructor;  
  const isItSelf = main === SelfReConstructor;  
  const Self = function () {  
    if (new.target) {  
      return new (isItSelf ? SelfReConstructor : main);  
    }  
    return Self;  
  };  
  Object.setPrototypeOf(Self, this);  
  Self.prototype.constructor = this;  
  return Self;  
};
```

ТО ЕСТЬ ПОТОМ МОЖНО ВОТ ТАК

```
const item = new SelfReConstructor();  
  
// item — это функция!  
item();           // callable ✓  
new item();       // newable ✓  
  
// и при этом...  
item instanceof SelfReConstructor; // true  
item instanceof Function;          // true  
item instanceof Object;             // true
```

`item` — одновременно и Тип и Интерфейс !

И ЕЩЁ ВОТ ТАК ...

```
const re = new item; // new item() работает!

console.log(re instanceof item);           // true ?!
console.log(re instanceof SelfReConstructor); // true
console.log(re instanceof Function);       // true
console.log(re instanceof Object);         // true

class MyInheritedItem extends re {
  /* yes, it works */
}
```

MyInheritedItem instanceof item

ЗАПУСТИМ И ПОСМОТРИМ

[▶ RUN ME | example](#)

```
const re = new item; // new item() работает!

console.log(re instanceof item);           // true ?!
console.log(re instanceof SelfReConstructor); // true
console.log(re instanceof Function);       // true
console.log(re instanceof Object);         // true

class MyInheritedItem extends re {
  /* yes, it works */
}
```

MyInheritedItem instanceof item

A КАК В TYPESCRIPT?

```
interface Self {  
    new (): Self;  
    value: number;  
}  
  
// Identity pattern: объект знает, как себя пересоздать  
interface Reconstructible<T> extends NewableFunction {  
    new (...args: any[]): T & Reconstructible<T>;  
    prototype: Reconstructible<T>;  
}  
  
// TS видит recursive тип  
const item: Self = new (Cstr as Reconstructible<Self>);  
const re: Self = new item;    // ok — item is constructible
```

ЗАЧЕМ САМО-ВОСПРОИЗВЕДЕНИЕ

ВОЗМОЖНОСТИ

- Пересоздать экземпляр с нуля
- Клонировать с новыми аргументами
- Undo / Redo: повторить создание

СИНТАКСИС

- `new myItem`
- каждый объект знает, как себя пересоздать
- `new myItem.MyReConstructibleField`
- удобно для типизированных полей

ЗАЧЕМ САМО-ВОСПРОИЗВЕДЕНИЕ

ВОЗМОЖНОСТИ

- Пересоздать экземпляр с нуля
- Клонировать с новыми аргументами
- Undo / Redo: повторить создание

СИНТАКСИС

- `new myItem`
- каждый объект знает, как себя пересоздать
- `new myItem.MyReConstructibleField`
- удобно для типизированных полей



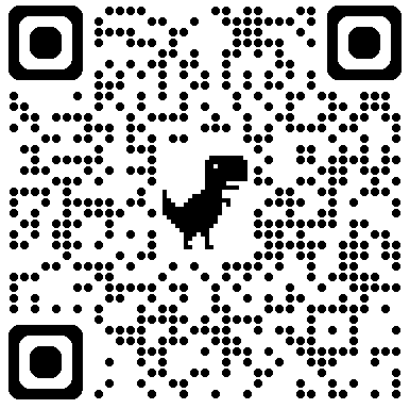
... ВОТ ТЕПЕРЬ ...



СПАСИБО



ССЫЛКИ



NPM

<https://www.npmjs.com/package/mnemonica>

GIST

<https://gist.github.com/wentout>

GITHUB

<https://github.com/wentout>



